Full Stack Development with MERN

Project Documentation

1. Introduction

- **Project Title:** Import Data using Transform Maps (Spreadsheet)
- **Team Members:**

S.No	Name	Role
1	Neha N	Team Member
2	Srimugi J	Team Lead
3	Sunil R	Team Member
4	Poovarasan K	Team Member

2. Project Overview

- **Purpose:** Organisations often keep data in spreadsheets (Excel or CSV files) that must be moved into a ServiceNow table. Entering each row by hand is slow and error-prone. This project imports spreadsheet data into ServiceNow in bulk using Import Sets and Transform Maps, so each column of the file lands in the right field of the target table automatically.
- **Features:** Key features and functionalities are listed below.
- **Spreadsheet upload:** Upload .xlsx / .csv files through System Import Sets > Load Data.
- **Staging table:** Raw rows are first held in an import set table, so live data is not touched until the transform runs.
- **Transform Map:** Source-to-target field mapping, with Auto Map Matching Fields support.
- **Coalesce:** A unique field (Employee ID) is coalesced so re-importing updates records instead of creating duplicates.
- **Import log and status:** Import set and transform history show how many records were inserted, updated, or ignored.

3. Architecture

- **Frontend:** The project is built on the ServiceNow platform, so the frontend is the standard ServiceNow UI (Next Experience) instead of React. The Application Navigator opens System Import Sets > Load Data, Transform Maps, Field Maps and the table list views. No custom front-end code is needed.
- **Backend:** The backend is the ServiceNow server-side engine instead of Node.js and Express.js. When a file is loaded, ServiceNow creates a Data Source and an Import Set, and each spreadsheet row becomes an Import Set Row in the staging table. When the transform runs, the Transform Map engine applies the field maps and the coalesce rule and inserts or updates records in the target table.

- **Database:** The database is the ServiceNow relational database instead of MongoDB. Two tables are used: a staging table (u_import_employee) and the target table Employee Details (u_employee_details).

Staging table: u_import_employee

Source column	Staging field	Description
Employee ID	u_employee_id	Unique ID of the employee (used for coalesce)
Name	u_name	Full name
Email	u_email	Email address
Department	u_department	Department name
Designation	u_designation	Job title

Target table: u_employee_details

Field label	Field name	Type
Employee ID	u_employee_id	String (coalesce field)
Name	u_name	String
Email	u_email	String
Department	u_department	String
Designation	u_designation	String

Data flow: Spreadsheet (.xlsx) > Data Source > Import Set Table (staging) > Transform Map > Target Table.

4. Setup Instructions

- **Prerequisites:** Software dependencies are listed below.
- A ServiceNow Personal Developer Instance (PDI) from developer.servicenow.com.
- A login with the admin role (to create tables, data sources and transform maps).
- A web browser such as Google Chrome or Microsoft Edge.
- Microsoft Excel (or any tool that can save .xlsx / .csv files).
- **Installation:** Step-by-step guide to set up the environment.

1. Sign in at developer.servicenow.com and request a Personal Developer Instance (there is nothing to clone, and no npm packages or environment variables are needed).
2. Open the instance URL and log in as admin.
3. Create the target table: System Definition > Tables > New. Set Label to Employee Details and Name to u_employee_details, then add the five fields listed in Section 3.
4. Create the spreadsheet Employee_Data.xlsx with the columns Employee ID, Name, Email, Department and Designation, with 5 to 10 sample records, and save it on your computer.

5. Folder Structure

- **Client:** ServiceNow has no React client folder. The user-facing parts of the project are listed below.

Component	Purpose
Employee_Data.xlsx	Source spreadsheet that holds the data to import.
System Import Sets > Load Data	Form used to upload the file and create the staging table.
Employee Details list view	Used to check the imported records.

- **Server:** ServiceNow has no Node.js server folder. The backend is organised as platform records, listed below.

Component	Purpose
Data Source	Stores the file type, format and sheet details of the uploaded spreadsheet.
Import Set (u_import_employee)	Staging table that holds the raw rows.
Transform Map	Links the staging table to the target table u_employee_details.
Field Maps	Define which source field fills which target field; Employee ID is marked Coalesce.
Transform History	Log of each transform run with inserted, updated and ignored counts.

6. Running the Application

- **Frontend:** Instead of npm start, open the instance in the browser and go to System Import Sets > Load Data.

5. Select Create table, enter the label Import Employee (name u_import_employee), choose Source of data = File and File format = Excel, upload Employee_Data.xlsx, and click Submit.
6. Click Create Transform Map on the Loaded Data page (or open System Import Sets > Transform Maps > New).
7. Set the target table to Employee Details [u_employee_details] and name the map Employee Import Map.
8. Click Auto Map Matching Fields, or add the field maps manually, and tick Coalesce for Employee ID.

- **Backend:** Instead of npm start in the server directory, the backend is started by running the transform.

9. Click Save, then click Transform and choose Employee Import Map.

10. Open the Employee Details table list and confirm all records appear; check Transform History for the inserted count.



7. API Documentation

Endpoints exposed by ServiceNow for this project (replace <instance> with your instance name):

Method	Endpoint	Description
POST	/api/now/import/u_import_employee	Insert a row into the staging table; the transform map runs automatically.
GET	/api/now/table/u_employee_details	Retrieve imported employee records.
GET	/api/now/table/u_employee_details/{sys_id}	Retrieve a single record by sys_id.
GET	/api/now/table/sys_import_set_run	View import / transform run history.

Example request (POST):

```
POST https://<instance>.service-now.com/api/now/import/u_import_employee
Content-Type: application/json
{
  "u_employee_id": "E101",
  "u_name": "Sample Name",
  "u_email": "sample@example.com",
  "u_department": "IT",
  "u_designation": "Developer"
}
```

Example response (values will differ in your instance):

```
{
  "import_set": "ISET0010001",
  "staging_table": "u_import_employee",
  "result": [{
    "transform_map": "Employee Import Map",
    "table": "u_employee_details",
    "display_name": "u_name",
    "display_value": "Sample Name",
    "record_link":
"https://<instance>.service-now.com/api/now/table/u_employee_details/
<sys_id>",
    "status": "inserted"
  }]
}
```

8. Authentication

- **Sessions:** Users sign in with a username and password, and ServiceNow keeps them logged in through a secure session.
- **Authorization:** Actions are controlled by roles. The admin role (or import_admin / import_transformer) is needed to create data sources, transform maps and run transforms.
- **Tokens:** REST calls use Basic Authentication or OAuth 2.0 bearer tokens, and each request is checked against the user's roles and the table Access Control Rules (ACLs).

9. User Interface

10. Testing

Testing strategy: manual testing in the ServiceNow Developer Instance, comparing the spreadsheet with the target table after each import. Tools used: list views, Transform History and Import Set Rows.

No.	Test case	Expected result	Status
1	Upload a valid .xlsx file	Rows are loaded into the staging table	Pass
2	Run the transform map	All rows are inserted into u_employee_details	Pass
3	Compare field values with the spreadsheet	Every column matches its target field	Pass
4	Re-import the same file (coalesce on Employee ID)	Existing records are updated, no duplicates created	Pass
5	Upload a file with an empty row	Empty row is ignored or flagged in the log	Pass

11. Screenshots or Demo

12. Known Issues

- A Personal Developer Instance goes to sleep after inactivity and must be woken from the developer site.
- Date or choice values that do not match the target field format may be skipped or stored incorrectly.
- Without a coalesce field, importing the same file twice creates duplicate records.
- If a spreadsheet column is renamed after the staging table is created, the mapping must be updated.

13. Future Enhancements

- Schedule imports automatically with a Scheduled Import.
- Add transform scripts (onBefore / onAfter) to clean and validate data during import.
- Import from other sources such as JDBC databases, REST APIs or LDAP.
- Send a notification when an import finishes or fails.
- Use reference-field mapping to link imported records to existing users and departments.